PLANO DE GERENCIAMENTO DE CONFIGURAÇÃO

Enfoque DevOps — Integração e Entrega Contínuas (CI/CD)

IMC-CALC — Calculadora de IMC (API em Python/FastAPI)

Versão 1.1

Campo	Descrição
Projeto	IMC-CALC — Calculadora de IMC (API)
Gerente de Projeto	Otavio Rodolfo Roecker
Gerente/Responsável de GCS/DevOps	Otavio Rodolfo Roecker
Revisor par (acesso de escrita)	@usuario-do-colega — colaborador com permissão Write
Fábrica de Software / Squad	Dupla: 1 desenvolvedor (autor) + 1 revisor par — Engenharia de Software, Católica SC
Repositório principal	https://github.com/otavioroecker/IMC-Calculadora (público)
Localização deste plano	docs/PGC_GCS_DevOps_IMC-CALC.pdf (item de configuração versionado no repositório)

Histórico de Revisões

Data	Versão	Descrição	Autor
23/09/2026	1.0	Elaboração do documento com enfoque em práticas DevOps (CI/CD), a partir do template da disciplina.	Otavio R. Roecker
28/10/2026	1.1	Atualização para refletir a implementação real: proteção da main, revisão por par com acesso de escrita, Environments staging/production com revisor obrigatório, pipelines ci.yml/cd.yml/rollback.yml, Dockerfile, quality gate de cobertura, publicação do plano no repositório.	Otavio R. Roecker

1. Introdução

Este Plano de Gerenciamento de Configuração (PGC) descreve as atividades de Gestão de Configuração de Software (GCS) executadas ao longo do ciclo de vida do produto IMC-CALC, com ênfase em práticas DevOps: Integração Contínua (CI), Entrega/Implantação Contínua (CD), Infraestrutura como Código (IaC) e automação de pipelines. As atividades identificam os itens de configuração, mantêm sua integridade e controlam sistematicamente as mudanças, com o mínimo de burocracia e o máximo de automação.

1.1 Finalidade

Estabelecer o padrão seguido pelos integrantes do projeto, garantindo rastreabilidade, versionamento e controle de mudanças, apoiado por pipelines automatizados de build, teste, integração e implantação.

1.2 Escopo

O plano cobre o sistema IMC-CALC: API RESTful em Python com FastAPI que calcula o Índice de Massa Corporal a partir de peso e altura e retorna a classificação (ex.: "Peso normal"). O sistema foi prototipado em C#/.NET e convertido para Python/FastAPI, mantido no repositório público indicado acima. O escopo abrange o código-fonte (app/), os testes (tests/), as dependências (requirements*.txt), os pipelines (.github/workflows), o empacotamento (Dockerfile, docker-compose.yml), os scripts (scripts/) e a documentação (README.md, docs/).

1.3 Definições, Acrônimos e Abreviações

Termo	Significado
GCS	Gestão de Configuração de Software
CI / CD	Integração Contínua / Entrega-Implantação Contínua
IaC	Infraestrutura como Código
CCM	Comitê de Controle de Mudanças
Baseline	Conjunto de itens de configuração em estado comprovado de estabilidade
Pipeline	Sequência automatizada de build, teste, análise e implantação
Branch protection	Regra do GitHub que impede push direto e exige PR, aprovação e checks na branch
Environment	Ambiente lógico do GitHub Actions (staging, production) com regras de proteção, como revisores obrigatórios
GitHub Flow	Estratégia de branching: main sempre implantável + branches de feature de vida curta
SemVer	Versionamento Semântico (MAJOR.MINOR.PATCH)
DORA Metrics	Métricas de desempenho DevOps (Deployment Frequency, Lead Time, Change Failure Rate, MTTR)
IMC / API	Índice de Massa Corporal / Application Programming Interface

1.4 Referências

- Documento base: Processos de Desenvolvimento de Software e a Integração com GCS.
- Template de PGC (RUP 7.0, IBM), adaptado pela disciplina Gerência, Configuração e Processos de Software, Católica SC, prof. Luiz C. Camargo.
- Repositório de referência (aula 1): https://github.com/lzcamargo/ongconecta
- Repositório do projeto: https://github.com/otavioroecker/IMC-Calculadora (README.md).

1.5 Visão Geral

A GCS é tratada como parte constante do pipeline de entrega: cada Pull Request revisado, testado e implantado gera evidências de rastreabilidade (logs de pipeline, aprovações de PR, tags de versão, imagens publicadas), reduzindo o esforço manual.

2. Cultura DevOps e Papéis na Gerência de Configuração

A responsabilidade é compartilhada entre desenvolvimento e operações. Como o projeto é pequeno, os papéis são consolidados entre o autor e o revisor par. A existência de um segundo integrante com acesso de escrita é requisito para que a regra de "1 aprovação" na branch main seja efetiva, já que o GitHub não permite que o autor aprove o próprio PR.

Papel	Quem exerce	Responsabilidades
Gerente de Projeto / Product Owner	Otavio R. Roecker	Prioriza o backlog (GitHub Issues), autoriza mudanças de maior impacto, acompanha métricas DORA.
Engenheiro DevOps / Release Engineer	Otavio R. Roecker	Mantém os workflows ci.yml, cd.yml e rollback.yml, o branching/versionamento e o registro de imagens (ghcr.io).
Desenvolvedor(a)	Otavio R. Roecker	Integra via PR de vida curta, escreve testes (pytest) e responde pela qualidade do que é implantado.
Revisor par (Write)	@usuario-do-colega	Revisa e aprova os PRs (obrigatório antes do merge); pode também propor PRs. Possui permissão Write no repositório.
SRE / Operações	Otavio R. Roecker	Acompanha logs, status de Actions e smoke tests; executa o rollback quando necessário.
QA / Automação de Testes	Otavio R. Roecker	Mantém tests/test_imc_service.py (unitários) e tests/test_api.py (API), além de scripts/acceptance_test.sh.
Segurança (DevSecOps)	Otavio R. Roecker	Dependabot, bandit e pip-audit no pipeline; segredos apenas em GitHub Secrets.
CCM	Professor orientador (simbólico)	Acionado em mudanças de alto risco; rotina segue o fluxo PR + pipeline.

3. Ferramentas, Ambientes e Infraestrutura

3.1 Ferramentas do Pipeline

Categoria	Ferramenta(s)	Finalidade
Controle de versão	Git + GitHub	Código, testes, pipelines e documentação versionados.
Orquestração CI/CD	GitHub Actions	ci.yml (PR), cd.yml (merge na main), rollback.yml (manual).
Empacotamento	Docker	Imagem imutável da API (python:3.12-slim, usuário não-root, HEALTHCHECK).
Registro de artefatos	GitHub Container Registry (ghcr.io)	Imagens versionadas: ghcr.io/otavioroecker/imc-calculadora:<tag>.
Dependências	pip + requirements.txt / requirements-dev.txt	Runtime separado das ferramentas de desenvolvimento.
Testes	pytest + pytest-cov + httpx	Unitários, testes da API (TestClient) e cobertura.
Análise estática	black + flake8	Formatação e lint obrigatórios no CI.
Segurança	bandit + pip-audit + Dependabot	Análise do código, auditoria e atualização das dependências, Actions e Docker.
Controle de acesso	Branch protection + Environments + CODEOWNERS	Bloqueio de push direto, revisão obrigatória e gate de produção.

Categoria	Ferramenta(s)	Finalidade
Rastreamento	GitHub Issues + templates	Solicitação de mudança e PR vinculados.

3.2 Ambientes e Infraestrutura

Os ambientes são implementados como **Environments do GitHub** (Settings > Environments), com regras de proteção distintas:

Ambiente	Finalidade	Proteção / gatilho de promoção
Desenvolvimento	Execução local (uvicorn --reload ou docker compose up).	Branch feature/*.
staging	Valida a imagem publicada e executa os testes de aceitação de POST /api/imc (scripts/acceptance_test.sh).	Automático após o merge na main e a publicação da imagem. Sem revisor.
production	Ambiente final; smoke tests (/docs e /api/imc), tag de baseline e release.	**Revisor obrigatório: Otavio R. Roecker.** O job de produção fica pausado até a aprovação em "Review deployments". Apenas a branch protegida (main) pode implantar.

Observação: por se tratar de uma API stateless acadêmica, staging e production executam a mesma imagem Docker em container no runner do GitHub Actions (ambientes lógicos). Para hospedagem pública, basta substituir o passo "Subir a imagem" do cd.yml por um deploy em PaaS (ex.: Render, Fly.io, Cloud Run), sem alterar o restante do fluxo.

3.3 Estratégia de Branching e Versionamento

- **main** — sempre implantável; protegida (ver 5.2).
- **feature/<issue>-<descrição>** — vida curta, uma por funcionalidade ou correção (ex.: feature/12-validacao-altura).
- **hotfix/<descrição>** — correções emergenciais, também via PR.
- Versionamento Semântico: o arquivo VERSION guarda MAJOR.MINOR (hoje 1.0); o PATCH é o número da execução do CD, gerando tags como v1.0.7. MAJOR/MINOR são alterados por PR. Mensagens de commit seguem Conventional Commits.

4. Identificação da Configuração

4.1 Itens de Configuração

- Código-fonte da API: app/main.py, app/routes.py, app/services.py, app/models.py.
- Testes: tests/test_imc_service.py e tests/test_api.py.
- Dependências: requirements.txt e requirements-dev.txt; configuração: pyproject.toml, .flake8, VERSION.
- Pipelines como código: .github/workflows/ci.yml, cd.yml, rollback.yml; .github/dependabot.yml; CODEOWNERS; templates de PR e Issue.
- Empacotamento: Dockerfile, docker-compose.yml, .dockerignore.
- Scripts: scripts/smoke_test.sh, acceptance_test.sh, setup_github.sh.
- Documentação: README.md, docs/adr/ e **este plano (docs/PGC_GCS_DevOps_IMC-CALC.pdf)**; evidências em docs/evidencias/.

4.2 Convenção de Nomenclatura

Item	Convenção	Exemplo
Branch de feature	feature/<issue>-<descrição>	feature/12-validacao-altura
Commit	Conventional Commits	feat(services): adiciona classificação de obesidade grau II
Tag de release	vMAJOR.MINOR.PATCH	v1.0.7
Imagem (commit)	ghcr.io/otavioroecker/imc-calculadora:sha-<7 chars>	...:sha-ff83f7e
Imagem (baseline)	ghcr.io/otavioroecker/imc-calculadora:<versão>	...:1.0.7

4.3 Estrutura do Repositório

```
.
|-- .github/
|   |-- workflows/ (ci.yml, cd.yml, rollback.yml)
|   |-- dependabot.yml, CODEOWNERS, PULL_REQUEST_TEMPLATE.md
|   `-- ISSUE_TEMPLATE/
|-- app/        (main.py, routes.py, services.py, models.py)
|-- tests/      (test_imc_service.py, test_api.py)
|-- scripts/    (smoke_test.sh, acceptance_test.sh, setup_github.sh)
|-- docs/       (PGC_GCS_DevOps_IMC-CALC.pdf, adr/, evidencias/)
|-- Dockerfile, docker-compose.yml, .dockerignore
|-- requirements.txt, requirements-dev.txt, pyproject.toml, .flake8
|-- VERSION, README.md, .gitignore
```

5. Integração Contínua (CI)

Todo Pull Request para a main dispara o workflow **ci.yml**. O check obrigatório "Qualidade e testes" precisa estar verde para o merge.

5.1 Etapas do Pipeline de CI

- Checkout e Python 3.12 com cache do pip; instalação de requirements-dev.txt.
- Formatação (black --check) e lint (flake8).
- Testes com pytest e cobertura mínima de 80% (--cov-fail-under=80).
- Segurança: bandit no código e pip-audit nas dependências.
- Job "Build da imagem Docker": constrói a imagem e executa smoke test (/docs e /api/imc).

5.2 Proteção da Branch main e Política de Revisão

Configuradas em Settings > Branches (automatizadas por scripts/setup_github.sh) e válidas inclusive para o administrador do repositório:

- **Push direto bloqueado:** toda mudança entra por Pull Request. Uma tentativa de git push origin main é recusada pelo GitHub (mensagem GH006/GH013); a evidência está em docs/evidencias/.
- **Revisão por par:** mínimo de 1 aprovação de @usuario-do-colega, colaborador com permissão **Write**. Aprovações obsoletas são descartadas após novos commits.
- Check de status "Qualidade e testes" obrigatório e branch atualizada com a main (strict).
- Resolução de conversas obrigatória, histórico linear, sem force-push e sem exclusão da main.
- "Include administrators" ativado: nem o dono do repositório ignora as regras.
- CODEOWNERS e template de PR padronizam a revisão; PRs pequenos e descritivos.

5.3 Critérios de Qualidade (Quality Gate)

- Cobertura >= 80% (resultado atual: 96%, com 100% em app/services.py).

- Zero erros de black/flake8; zero achados do bandit; zero vulnerabilidades conhecidas no pip-audit.
- Zero alertas críticos/altos do Dependabot antes do merge.

6. Entrega e Implantação Contínua (CD)

Após o merge, o workflow **cd.yml** constrói e publica a imagem uma única vez e promove essa mesma imagem, sem recompilação, pelos ambientes.

6.1 Etapas do Pipeline de CD

- Build e publicação da imagem no ghcr.io (tag sha-<commit>).
- **staging**: execução da imagem e testes de aceitação de POST /api/imc.
- **Gate de aprovação**: o environment production exige aprovação do revisor obrigatório (Otavio R. Roecker).
- **production**: execução da imagem aprovada e smoke tests pós-deploy.
- Baseline: imagem marcada com a versão SemVer, tag vX.Y.Z e release com notas automáticas.

6.2 Estratégias de Implantação

Estratégia	Descrição	Uso no IMC-CALC
Rolling update	Substituição gradual das instâncias.	Padrão, ao migrar para PaaS de container único.
Blue-Green	Duas versões coexistem; tráfego comutado de uma vez.	Não utilizada nesta fase.
Canary	Fração do tráfego para a nova versão.	Evolução futura.

6.3 Rollback e Recuperação

O workflow **rollback.yml** (disparo manual, parâmetro "versao") reimplanta uma imagem anterior já publicada no ghcr.io. Também passa pelo gate do environment production e executa smoke tests ao final.

7. Infraestrutura como Código (IaC)

Dockerfile, docker-compose.yml, workflows e as regras do repositório (scripts/setup_github.sh) são versionados e alterados somente por PR. A configuração dos Environments e da proteção da main é reproduzível pelo script, evitando configuração manual não rastreada. Terraform é dispensado nesta fase.

Melhoria futura: ambientes efêmeros (review app) por Pull Request, registrados no backlog.

8. Controle de Configuração e Mudança

8.1 Processo de Solicitação de Mudança

Issue (template "Solicitação de mudança") -> branch feature -> Pull Request -> CI -> aprovação do revisor par -> merge -> CD (staging -> aprovação -> production). Esse fluxo é o registro formal e rastreável da mudança.

8.2 Comitê de Controle de Mudança (CCM)

Papel simbólico exercido pelo professor orientador, acionado em mudanças de alto impacto (ex.: alteração do contrato de POST /api/imc ou da arquitetura). Em situações rotineiras, a revisão por par e o pipeline substituem o comitê.

9. Baselines e Releases

Cada implantação aprovada em produção gera uma baseline: tag vX.Y.Z associada ao commit e à imagem Docker de mesma versão. As notas de versão são geradas automaticamente a partir dos commits/PRs desde a última tag e publicadas na aba Releases.

10. Estimativa do Status de Configuração

10.1 Monitoramento e Observabilidade

Logs do Uvicorn, HEALTHCHECK da imagem, smoke tests do pipeline e status das execuções no GitHub Actions.

10.2 Métricas DORA

Métrica	Descrição
Deployment Frequency	Frequência de implantações em produção (releases/tags).
Lead Time for Changes	Tempo entre o commit/PR e a implantação em produção.
Change Failure Rate	Percentual de implantações com falha ou rollback.
MTTR	Tempo médio de restauração (ex.: via rollback.yml).

10.3 Relatórios e Auditorias

O histórico de PRs (aprovações), execuções do Actions (inclusive aprovações de deployment), commits, tags e releases é o insumo de auditoria. As evidências de conformidade (push recusado na main, proteção da branch, revisor com escrita, Environments e revisor obrigatório) ficam em docs/evidencias/.

11. Marcos

- Plano de GCS publicado no repositório (docs/).
- Proteção da main configurada e push direto recusado (evidenciado).
- Revisor par (@usuario-do-colega) com acesso de escrita e primeiro PR aprovado.
- Pipeline de CI em operação (ci.yml) como check obrigatório.
- Environments staging e production criados, este com revisor obrigatório.
- Containerização concluída e imagem publicada no ghcr.io.
- Primeira release automatizada (v1.0.x) após aprovação do gate de produção.

12. Treinamento e Recursos

Consolidar conhecimento em:

- FastAPI e Pydantic; pytest e pytest-cov.
- GitHub Actions (YAML), Environments e branch protection.
- Docker (build, run, publish) e GitHub Container Registry.
- Conventional Commits e SemVer.
- Recursos: documentação oficial, material da disciplina e apoio do professor.

13. Controle de Software de Terceiros

Não há subcontratados. As bibliotecas (FastAPI, Uvicorn, pytest, httpx, entre outras) são itens de configuração declarados em requirements.txt (runtime) e requirements-dev.txt (desenvolvimento), com intervalos de versão controlados. O Dependabot (pip, GitHub Actions e Docker) abre PRs de atualização que seguem o mesmo fluxo de revisão e CI; o pip-audit verifica vulnerabilidades a cada PR.

14. Quadro-Resumo do Pipeline CI/CD

Etapa	Descrição	Gatilho / controle
CI	black, flake8, pytest (>=80%), bandit, pip-audit, build e smoke test Docker.	Abertura/atualização de Pull Request
Merge	Integração na main. Push direto recusado.	1 aprovação do revisor par (Write) + check "Qualidade e testes" verde
CD - Build	Publicação da imagem no ghcr.io.	Merge na main
CD - Staging	Deploy + testes de aceitação de /api/imc.	Automático (environment staging)
Gate de Release	Aprovação manual para produção.	Revisor obrigatório do environment production
CD - Produção	Deploy, smoke tests, tag/baseline e release.	Após aprovação do gate
Rollback	Reimplantação de versão anterior.	Manual (workflow_dispatch) + gate de production

Este plano deve ser revisado conforme o projeto evoluir, mantendo os quatro pilares da GCS — identificação, controle de versão, controle de mudanças e auditoria/status — apoiados por automação.